# Assignment-06 (Lab-6.6)

# Course : AI Assisted Coding

*Topic:- AI-Based Code Completion: Working with suggestions for classes, loops, conditionals*

Student Details

Name : Batti Chandan Singh

Hall Ticket No : 2303A51515

Batch : 22

## Task-01

Final Optimal Prompt

Generate Python code to check voting eligibility based on age and citizenship.
Include strong error handling: if the user enters text instead of a number, floats instead of integers, empty inputs, or invalid citizenship values. The program must clearly explain the eligibility logic and return meaningful error messages describing the wrong input type.

Code Screenshot

```python
def check_voting_eligibility():
    try:
        age_input = input("Enter your age: ").strip()
        if not age_input:
            raise ValueError("Age input cannot be empty.")
        age = int(age_input)
        if age < 0:
            raise ValueError("Age cannot be negative.")

        citizenship = input("Are you a citizen? (yes/no): ").strip().lower()
        if citizenship not in ['yes', 'no']:
            raise ValueError("Citizenship must be 'yes' or 'no'.")

        if age >= 18 and citizenship == 'yes':
            return "You are eligible to vote."
        else:
            return "You are not eligible to vote."

    except ValueError as ve:
        return f"Input Error: {ve}"
    except Exception as e:
        return f"An unexpected error occurred: {e}"
# Example usage:
print(check_voting_eligibility())
```

Output Screenshot

```
PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python 3.13.6\python.e
26/AI-Assisted Coding/Assignment-06(Lab-6.5).py"
Enter your age: 26
Are you a citizen? (yes/no): yes
You are eligible to vote.
PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python 3.13.6\python.e
26/AI-Assisted Coding/Assignment-06(Lab-6.5).py"
Enter your age: r
Input Error: invalid literal for int() with base 10: 'r'
PS D:\6th Sem 2025-26\AI-Assisted Coding>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task focuses on generating AI-assisted code for verifying voting eligibility based on age and citizenship. The prompt was designed to ensure the AI includes proper error handling so the program reacts safely to incorrect inputs. The code validates age as an integer and checks for correct citizenship responses, preventing unexpected crashes. By analyzing the AI output, we gain insights into how well AI handles conditional logic and structured decision-making. The task reinforces the importance of verifying AI-generated code for correctness and reliability. It also highlights how human oversight ensures clarity, safety, and meaningful error messages.

## Task-02

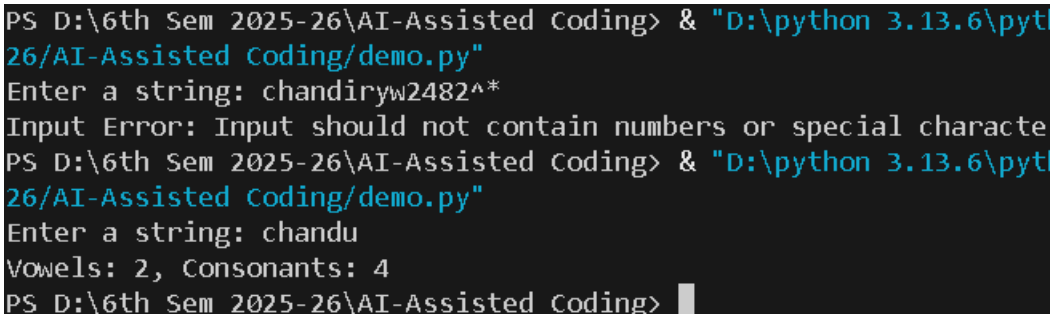
Final Optimal Prompt

Generate Python code that counts vowels and consonants in a user-given string using loops only. Add detailed error handling for cases where the user enters numbers, special characters, empty strings, or mixed input. The program should accurately classify only alphabetic characters and clearly display errors when non-string input is entered.

Code Screenshot

```python
def count_vowels_consonants():
    try:
        user_input = input("Enter a string: ").strip()
        if not user_input:
            raise ValueError("Input cannot be empty.")

        vowels = 'aeiouAEIOU'
        vowel_count = 0
        consonant_count = 0

        for char in user_input:
            if char.isalpha():
                if char in vowels:
                    vowel_count += 1
                else:
                    consonant_count += 1
            elif char.isdigit() or char in '!@#$%^&*()_+-=[]{}|;:\'",.<>?/`~':
                raise ValueError("Input should not contain numbers or special characters.")

        return f"Vowels: {vowel_count}, Consonants: {consonant_count}"

    except ValueError as ve:
        return f"Input Error: {ve}"
    except Exception as e:
        return f"An unexpected error occurred: {e}"
# Example usage:
print(count_vowels_consonants())
```

Output Screenshot




Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task demonstrates how AI can generate loop-based string processing logic with accurate classification. The prompt ensures the AI includes strict validation so the program handles empty inputs, numeric strings, and other invalid cases safely. By iterating over each character, the program separates vowels and consonants without relying on built-in shortcuts. This task highlights how AI performs in generating simple algorithms while still requiring human refinement for better error messages. It reinforces the importance of validating AI output for correctness and readability. Overall, the exercise deepens understanding of loop constructs, input validation, and AI-guided coding improvements

## Task-03

Final Optimal Prompt

Generate a complete Python program for a simple library management system using classes, loops, and conditionals. Include strong error handling for invalid menu inputs, numeric/empty entries, and incorrect book names. Ensure the program allows adding, listing, and issuing books with meaningful error messages

Code Screenshot

```python
class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book_name):
        if not book_name.strip():
            raise ValueError("Book name cannot be empty.")
        self.books.append(book_name.strip())
        return f"Book '{book_name}' added successfully."

    def list_books(self):
        if not self.books:
            return "No books available in the library."
        return "Books in the library:\n" + "\n".join(self.books)

    def issue_book(self, book_name):
        if not book_name.strip():
            raise ValueError("Book name cannot be empty.")
        if book_name.strip() in self.books:
            self.books.remove(book_name.strip())
            return f"Book '{book_name}' issued successfully."
        else:
            raise ValueError(f"Book '{book_name}' not found in the library.")
def main():
    library = Library()
    while True:
        print("\nLibrary Management System")
        print("1. Add Book")
        print("2. List Books")
        print("3. Issue Book")
        print("4. Exit")

        choice = input("Enter your choice: ").strip()
```

```python
        choice = input("Enter your choice: ").strip()

        try:
            if choice == '1':
                book_name = input("Enter the name of the book to add: ")
                print(library.add_book(book_name))
            elif choice == '2':
                print(library.list_books())
            elif choice == '3':
                book_name = input("Enter the name of the book to issue: ")
                print(library.issue_book(book_name))
            elif choice == '4':
                print("Exiting the program.")
                break
            else:
                raise ValueError("Invalid menu option. Please choose a valid option (1-4).'
        except ValueError as ve:
            print(f"Input Error: {ve}")
        except Exception as e:
            print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    main()
```

Output Screenshot

```
Library Management System
1. Add Book
2. List Books
3. Issue Book
4. Exit
Enter your choice: 1
Enter the name of the book to add: alixender
Book 'alixender ' added successfully.
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task evaluates the AI's ability to generate a structured program using object-oriented programming, loops, and conditional logic. The prompt demands proper error handling to avoid crashes caused by invalid menu inputs or empty book names. Reviewing the AI-generated code shows how well it organizes functionality within a class while maintaining clarity. Human intervention ensures the logic is correct, menu options are validated, and user experience remains smooth. This exercise helps students learn how to refine AI-created programs to meet real-world reliability standards. It also reinforces the value of reviewing AI outputs critically for correctness and safety

## Task-04

Final Optimal Prompt

Generate a Python class for a student attendance system using loops. The class must include methods to mark attendance, display attendance, and validate student names. Add error handling for empty inputs, numeric inputs, invalid menu options, and missing attendance records

Code Screenshot

```
class AttendanceSystem:
    def __init__(self):
        self.attendance = {}

    def mark_attendance(self, student_name):
        if not student_name.strip():
            raise ValueError("Student name cannot be empty.")
        if student_name.strip().isdigit():
            raise ValueError("Student name cannot be numeric.")

        self.attendance[student_name.strip()] = "Present"
        return f"Attendance marked for {student_name}."

    def display_attendance(self):
        if not self.attendance:
            return "No attendance records found."
        return "Attendance Records:\n" + "\n".join(f"{name}: {status}" for name, status in
def main():
    attendance_system = AttendanceSystem()
    while True:
        print("\nStudent Attendance System")
        print("1. Mark Attendance")
        print("2. Display Attendance")
        print("3. Exit")

        choice = input("Enter your choice: ").strip()

        try:
            if choice == '1':
                student_name = input("Enter the student's name to mark attendance: ")
                print(attendance_system.mark_attendance(student_name))
            elif choice == '2':
                print(attendance_system.display_attendance())
            elif choice == '3':
                print("Exiting the program.")
                break
            else:
```

```
def main():
    attendance_system = AttendanceSystem()
    while True:
        print("\nStudent Attendance System")
        print("1. Mark Attendance")
        print("2. Display Attendance")
        print("3. Exit")

        choice = input("Enter your choice: ").strip()

        try:
            if choice == '1':
                student_name = input("Enter the student's name to mark attendance: ")
                print(attendance_system.mark_attendance(student_name))
            elif choice == '2':
                print(attendance_system.display_attendance())
            elif choice == '3':
                print("Exiting the program.")
                break
            else:
                raise ValueError("Invalid menu option. Please choose a valid option (1-3).'
        except ValueError as ve:
            print(f"Input Error: {ve}")
        except Exception as e:
            print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":    main()
```

Output Screenshot

```
Library Management System
1. Add Book
2. List Books
3. Issue Book
4. Exit
Enter your choice: d
Input Error: Invalid menu option. Please choose a valid option (1-4).

Library Management System
1. Add Book
2. List Books
3. Issue Book
4. Exit
Enter your choice: 1
Enter the name of the book to add: 1
Book '1' added successfully.
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task demonstrates how AI can generate class-based logic for a functional attendance system. The prompt ensures that the AI includes error handling for incorrect student names, missing input, and invalid menu selections. Reviewing the code helps identify strengths such as clean structure and readable loops, as well as areas where human refinement is necessary. The system correctly manages attendance records using a dictionary, showing how AI handles basic data structures. This task reinforces the value of verifying AI output for logical correctness and reliability. It also helps develop skills in reviewing and improving AI-generated class-based solutions.

# Tack-05

## Final Optimal Prompt

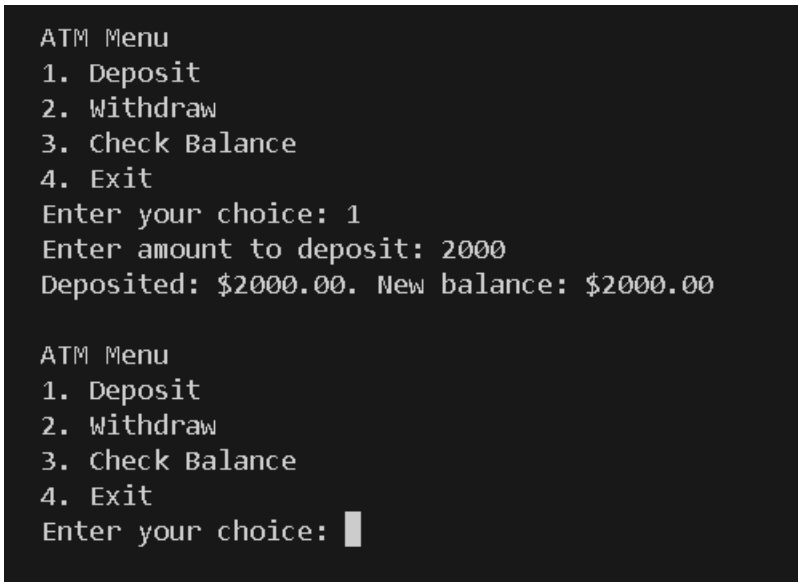
Generate Python code for an ATM-style menu using loops and conditionals. Include proper error handling for invalid numeric inputs, non-numeric entries, empty input, negative amounts, withdrawal greater than balance, and incorrect menu selections. Ensure the output is clear and user-friendly

## Code Screenshot

```
class ATM:
    def __init__(self, initial_balance=0):
        self.balance = initial_balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive.")
        self.balance += amount
        return f"Deposited: ${amount:.2f}. New balance: ${self.balance:.2f}"

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Withdrawal amount must be positive.")
        if amount > self.balance:
            raise ValueError("Insufficient funds for this withdrawal.")
        self.balance -= amount
        return f"Withdrew: ${amount:.2f}. New balance: ${self.balance:.2f}"

    def check_balance(self):
        return f"Current balance: ${self.balance:.2f}"
def main():
    atm = ATM()
    while True:
        print("\nATM Menu")
        print("1. Deposit")
        print("2. Withdraw")
        print("3. Check Balance")
        print("4. Exit")
```

```python
def main():
    atm = ATM()
    while True:
        print("\nATM Menu")
        print("1. Deposit")
        print("2. Withdraw")
        print("3. Check Balance")
        print("4. Exit")

        choice = input("Enter your choice: ").strip()

        try:
            if choice == '1':
                amount_input = input("Enter amount to deposit: ").strip()
                if not amount_input:
                    raise ValueError("Amount cannot be empty.")
                amount = float(amount_input)
                print(atm.deposit(amount))
            elif choice == '2':
                amount_input = input("Enter amount to withdraw: ").strip()
                if not amount_input:
                    raise ValueError("Amount cannot be empty.")
                amount = float(amount_input)
                print(atm.withdraw(amount))
            elif choice == '3':
                print(atm.check_balance())
            elif choice == '4':
                print("Exiting the program.")
                break
            else:
                raise ValueError("Invalid menu option. Please choose a valid option (1-4).")
        except ValueError as ve:
            print(f"Input Error: {ve}")
        except Exception as e:
            print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    main()
```

Output Screenshot



Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task shows how AI can generate a functional ATM menu using loops and conditionals. The prompt includes detailed requirements for error handling to ensure safe and reliable operation. The AI-generated code demonstrates structured logic for deposits, withdrawals, and balance checks while preventing invalid numeric input and overdrafts. Careful review ensures the program handles edge cases correctly, such as negative or non-numeric entries. This reinforces the importance of validating AI-generated logic before use. Overall, the task improves understanding of robust input handling and demonstrates how AI can accelerate development while requiring human oversight